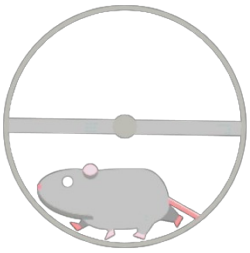


COM-304 Software Radios & Radars

Final Project Report

Where are you Jerry?

Through-wall Detection of Small Rodents using mmWave Radar

Design and implementation of a system leveraging mmWave radar to detect the motion of small moving targets behind non-radar-transparent objects.

Authors

Jan van der Kuyl
Kasper Kelly
Kerim Tuncelli

Institution

École Polytechnique Fédérale de Lausanne (EPFL)

May 2026

Contents

1 Introduction

1.1 Problem Statement

This project endeavours to investigate whether mmWave radar sensing techniques can be used to detect rodent movement within building walls in a non-invasive and scalable manner.

1.2 Background

In many places around the world, there are serious problems with rodents living in the walls of buildings. This is a genuine issue for many reasons, due to the damage caused and the risk of disease transfer which is often correctly attributed to the presence of small rodents, such as rats. Research from Byers et al[1] confirms these problems caused by rats, and that their activity during periods of quiet life makes catching them a difficult and expensive process.

Modern techniques in reducing rodent infestations have been applied in recent years, more recently making use of camera traps and machine learning through environmental studies[2]. However, these techniques fall victim to line-of-sight requirements and lighting restrictions and also involve more intense set-up requirements and more often than not, intensive human in the loop intervention.

Others have been leveraging smart traps and IR sensors[3], but IR relies on detecting the heat of an object, something which would not be possible through the walls of a building, especially when tracking rodents on the move or those behind thicker materials.

There has been work done before using LoRa radio signals for rodent tracking[4], using mmWave radar to monitor[5] rats in cages and also bioradar for biosensing of rodents in controlled environments[6].

Although these alternative sensing techniques have been explored in controlled environments, there remains a lack of practical, non-invasive systems capable of reliably detecting rodent movement inside building walls. Therefore, a new approach is needed that can monitor rodent activity through structural materials without requiring line-of-sight, extensive installation or continuous human intervention.

While there has been some research done into non-invasively tracking and biosensing rodents using radar, to our knowledge there has been no research done in using mmWave Radar to track rodents through wall. This makes the scope of this project extremely exciting for us due to its novelty.

1.3 Project Goals

As outlined in the project proposal, we planned to meet the following deliverables.
Baseline Deliverables:

1. Achieve satisfactory results in terms of clarity of the image (i.e. have a video of the position of the rodent moving through the wall) and computational efficiency of the proposed algorithms using pre-recorded data.

Expected Deliverables:

1. In addition to the baseline deliverables, achieve satisfactory results when conducting the experiment in real-time.

Ideal Deliverables:

1. Implement an algorithm to recognize the rodent from other moving objects within the infrastructure by implementing an classification task.
2. "Best Ideal Case" : Being able to create a 'map' of the pipes through walls (i.e. locate pipes that have water flowing through them making use of the properties of wave propagation through that particular medium).

1.4 Final System

Our final system met the Baseline, Expected and Ideal deliverables we set out to achieve in our project proposal. We developed a system capable of consistently detecting rats in pipes and through walls.

The so called "best-ideal-case" was not met due to time restrictions towards the end of the project. However, we believe that with a non-substantial amount of work, the pipeline we developed could be applied to track water moving through pipes following the same principles

2 System Overview

2.1 Hardware Setup

Two main hardware configurations were considered during the project: a single-radar setup using Windows 11 and mmWave Studio, and a dual-radar setup using Linux. Each configuration has distinct advantages and limitations, as described below.

2.1.1 Configuration 1: Single Radar using Windows 11 and mmWave Studio

The first configuration consists of a single radar sensor connected to a Windows 11 machine running mmWave Studio. In this setup, the radar produces a raw input data stream, which is then processed using the pipeline described in the following sections.

A key advantage of this configuration is that the required backbone architecture was already available, making the initial setup relatively straightforward to build upon. As a result, this configuration provided a reliable starting point for data acquisition, signal processing, and initial system validation. It was also the configuration that underwent the most extensive testing with the radar and rat-detection test stand. Furthermore, this setup was used for the final demonstration of the project.

However, this configuration also presents several limitations. Most importantly, it supports only a single radar sensor, which restricts the spatial coverage and limits the potential accuracy of the system. In addition, the Windows-based processing pipeline was found to be comparatively slow, which constrained real-time performance during experimentation.

2.1.2 Configuration 2: Dual Radar on Linux

The second configuration consists of two radar sensors operating simultaneously on a Linux-based machine. In this setup, each radar produces a raw input stream, which is processed using a similar pipeline to the single-radar configuration. The local beamforming outputs from each

radar can then be transformed from their respective local polar coordinate systems into a shared global Cartesian coordinate space.

The main advantage of this configuration is its ability to support multiple radar sensors. By combining measurements from two radars, the system can increase spatial coverage and potentially improve detection and localization accuracy. In addition, the Linux-based implementation offers significantly faster processing compared to the Windows-based setup, making it more suitable for real-time or near-real-time applications.

Despite these advantages, this configuration was more challenging to implement. In particular, the available documentation for operating multiple radars simultaneously on a Linux machine was limited. Consequently, additional effort was required to configure the hardware, synchronize the data streams, and integrate the outputs into a common processing framework.

2.2 Software Pipeline

From a high level perspective, the software pipeline involved 5 key steps to process the raw input stream from the radar into interpretable visualisations.

The first step was range gating of the input stream. This method involves only considering radar input from selected bins within the predetermined anticipated distance to the wall. We ended up restriction our range between 0 and 1.5 meters. We experimented with this technique and found that it greatly helped reduce false positives downstream at the classification task. We believe that future work could be done here to automatically determine the position of the wall from the radar so that these parameters could be set automatically without manual input.

The second part of the pipeline is associated with removal of as much noise as possible, again to help the classification task downstream not to generalise to unwanted noise. Techniques such as Constant False alarm Detection (CFAR), Signal to Noise Ratio (SNR), noise flooring, hamming windows and many others were used in this stage. The theory behind these techniques and an explanation of their implementation can be found in the sections below.

The third section of the pipeline made use of the doppler effect to help identify areas of signal which contained actively moving elements above a specified threshold. This was tuned to literature data of speeds of regular rats[7]. The implementation of this movement tracking too is later explained in detail.

The penultimate part of the processing pipeline involved the classification task. This was used in order to attempt to differentiate movement of a rat from typical noisy radar signals, signals too large to be those of a small rodent and signals from stationary objects. The algorithm developed for this task is discussed at length in sections below.

The final step of the pipeline as to visualise the output of our system. The output can be separated into 3 different modalities. The first is a a beamformed output which is useful for purpose of debugging and testing. The second is an alarm-style box which outputs the result of the classifier. This can be seen in 9 and 10 in the bottom left corner and and is green for non-detection and red for positive detection of a rodent. The final output is the GUI frontend we developed, this can be seen in 8 and is discussed in further detail later.

A visualisation of this pipeline can be seen in Figure 1.

2.3 Experimental Setup

The experimental setup was designed to emulate a real physical wall and real motion of rats. We were unable to use real rats, and so two primary test targets are used: “Jerry”, a stuffed

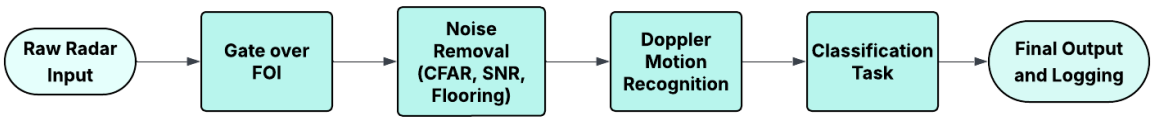


Figure 1: Block diagram outlining the full software pipeline from raw input data to output visualisations

animal rat attached to a string, and "Remi", a wind-up cat toy. These fake decoys, especially remi, allow repeatable motion patterns and easy testing of our system. They can both be seen in Figures 2 and 3

Figure 2: Remi, the windup rat

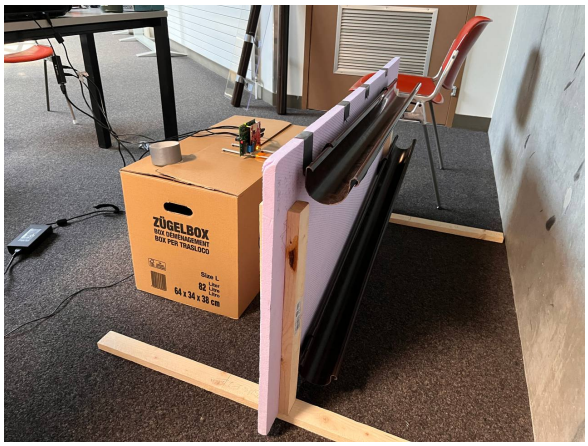
Figure 3: Jerry, plushy on a string

A modular test stand is used to simulate wall structures and constrained environments. The stand includes interchangeable pipes and gutters through which the targets can be moved. This modularity enables systematic testing under different geometric constraints and motion corridors.

To emulate different wall conditions, a range of materials are used including foam, wood and acrylic, as well as combinations of these materials. In particular, layered configurations such as wood combined with insulation foam are used to approximate realistic building walls as can be seen in Figures 4a and 4b.

The mechanical stability of the setup was a key design consideration such that vibrations are minimized to reduce noise and prevent false radar detections caused by unintended structural motion or noise introduced into the sensing environment.

(a) Image of the setup from the radar's perspective



(b) Image of the setup facing the radar

Figure 4: Foam and wood layered test setup from two perspectives

3 Detailed System Description

3.1 Final Radar Configuration

Table 1 outlines the final parameters used for the radar.

Parameter	**Value**	**Notes**
NUM_TX	3	Number of active transmitters.
NUM_RX	4	Number of active receivers.
START_FREQ	77 GHz	Starting frequency of the FMCW chirp.
IDLE_TIME	7 μs	Time interval between consecutive chirps.
RAMP_END_TIME	63.99 μs	Duration of the frequency ramp.
ADC_START_TIME	9.02 μs	Time at which ADC sampling begins.
FREQ_SLOPE	62.474 MHz/μs	Frequency slope of the FMCW chirp.
ADC_SAMPLES	128	Number of ADC samples collected per chirp.
SAMPLE_RATE	2420 ksps	ADC sampling rate.
RX_GAIN	36 dB	Receiver gain used to improve weaker radar returns.
START_CHIRP_TX	0	Index of the first chirp in the frame configuration.
END_CHIRP_TX	2	Index of the last chirp, corresp. to NUM_TX − 1.
CHIRP_LOOPS	32	Number of repetitions of the chirp sequence per frame.
NUM_FRAMES	0	Enables continuous streaming mode.
PERIODICITY	40 ms	Time period between consecutive frames.

Table 1: Final radar configuration parameters

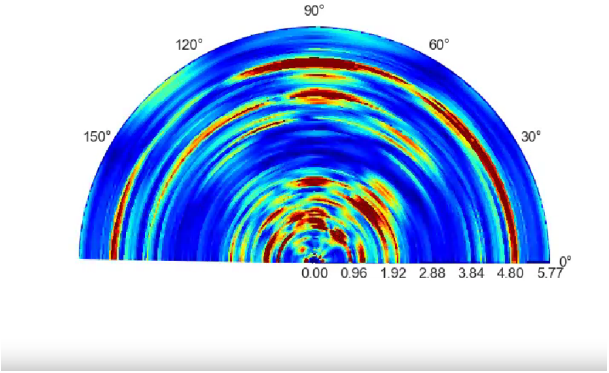


Figure 5: Example Noisy Beamformed Output

3.2 Processing Pipeline

This section will describe the methods used to interpret the noisy incoming signals into interpretable data for our classification tasks. Some of the methods used were derived from prior code given to us in tutorial tasks.

3.2.1 Hamming Window

The first step is to take the raw radar data that has already been a result of the tx and rx mixing, and apply a hamming window to it. Hamming window is a necessary first step because we then take the fft of along the range dimension of the data. Fft's are meant to be down on single period's of data, not messy real world data. The hamming window fixes this by smoothing out the edges to zero to resemble a repeating signal. It uses the following equation for each of N adc samples: $w[n] = 0.54 - 0.46cos(\frac{2\pi n}{N-1}), n = 0, 1, ..., N-1$. This give us a smoothed out bell curve which we then take the fft of.

3.2.2 Fast Fourier Transform

The range fft takes an fft of the data from the hamming window across the range spectrum, creating an array of range bins indicating the magnitude at each unit distance of $\frac{c*f_s}{2*slope*N}$ from the radar. This gives us a beamformed output, like that of Figure 5, but still needs work to look clean.

3.2.3 CFAR

Constant False Alarm Rate is one of our many tools for filtering out noise. We didn't use it at first even though it was used in the previous repository, but we quickly found it helped remove side lobing from objects. Basically the rat would show up across the entire arc, rather than a single splotch like we wanted. It works by convolving a window over every cell and comparing the cell's power to the average of its neighbors. Cells that significantly beat their local neighborhood pass as detections; cells that are bright but no brighter than the region they sit in get rejected. This helped a lot in making our beamform images stand out.

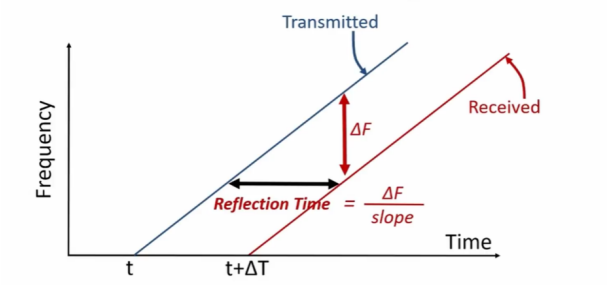


Figure 6: Diagram of a Radar Chrip

3.2.4 SNR (Signal to Noise Ratio)

Some extra noise filtering techniques we used were sound noise ratio thresholds as well as a flat out noise floor threshold. The SNR thresholding essentially cuts out all values that aren't greater than 2x the median magnitude across the entire heatmap. In our case, the median is never going to be anything more than some light noise, so this helps us get rid of a lot of it.

3.3 Detection Algorithms

After all of these filtering processes, we get a much cleaner beamform graph. However, the area we are interested in is inside the pipe, which shows up as a solid red arc. When a mouse moves through the pipe, it doesn't add anything to the beamform, since that area is already red. We decided to investigate a method that would only display movement on our beamform, which led to us using the doppler effect.

3.3.1 Doppler Effect

The first part of the detection algorithms this project employed was to use the doppler effect produced by moving targets to single out radar signals from only target in motion. This assisted the classification algorithm explained later to not overfit to noise in the gathered input. The doppler detection works by taking the fft of the range fft in respect to chirp loops. Chirps are sets of signals that increase in frequency overtime, allowing us to determine the distance of objects.

Chirp loops are sets of chirps as seen in Figure 6, that can be used to extract velocity data. Our configuration used 32 chirp loops and a periodicity of 40ms, which is the time between each set of chirp loops. The reason taking the FFT across chirp loops recovers velocity is that a moving target's signal phase progresses slightly from one chirp to the next, by $\Delta\varphi = \frac{(4\pi * v_r * T_c)}{\lambda}$ where v_r is the target's radial velocity, T_c is the time between chirps, and $\lambda \approx 3.9$ mm at the 77 GHz center frequency. After centering the velocity FFT around 0, we can then mask velocity values near 0, only keeping positive and negative velocity values that could represent our mouse. An example of this can be seen in Figure 7

3.3.2 Novel "Jerry Classifier"

One of the major problems in detecting small animals using radar is that the small amount of noise they produce when moving is quickly overshadowed by noise from any other large structures or from readings of large moving objects that have reflected into the radar's FOV.

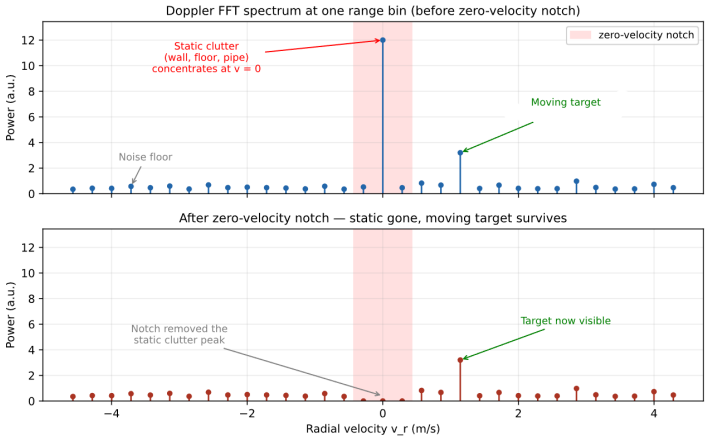


Figure 7: Doppler FFT example

Thus, a score based heuristic classifier was developed for this project in order to identify and detect the presence of a small, moving target within a region of interest. Design of this algorithm was supported by the plethora of data gathered throughout investigation.

The detector takes in a one dimensional beamformed input which consists of the radar signals of varying strengths across range bins. In design, the goal of this algorithm was to be robust to surrounding noise, stable for false positives in certain radar frames and with sufficiently low computational complexity such that it can be applied to our system in real time.

The first step of the algorithm, like the displayed bf output for the user, is to make use of a subset of range bins within the region of interest (ROI). For this project, the region of interest is defined as the wall and pipe network behind it, and as such the range bins were defined to cover a range for this ROI with sufficient margin. These parameters were defined and tuned with testing, and provided the radar was setup at consistent distances from the wall, proved effective across differing setups.

Next, we perform adaptive noise thresholding. That being we attempt to determine whether or not there is significant motion/reflections within the ROI and as such we compute an adaptive threshold based on surrounding noise for each radar frame. To do this, we first ignore the noise from within the ROI by ignoring input from range bins within this region:

$$r\_idxs_{noise} = r\_idxs - r\_idxs_{ROI}$$

$$BF_{noise} = BF \text{ such that } r\_idxs = r\_idxs_{noise}$$

Then, a noise floor is calculating by considering the mean magnitude of all non-zero bins outside the ROI. Also, where there existed a range bin $= 0$ due to previous pre-processing in other parts of the algorithm, we set this value to 1 before taking the mean to avoid skewing this noise threshold.

$$T_{noise} = \frac{1}{N} \sum_{i=1}^{N} B_{noise,i}$$

And the threshold is given by

$$T_{detect} = \alpha \cdot T_{noise}$$

Where α defines a noise sensitivity that was tuned to our setup. This part of the classifier pre-processing allows the classier to automatically adapt to changing setups with differing background noise levels, and constant setups with changing environmental noise.

After considering background noise, we then look to detect rats in each frame. Basically, for each radar frame, the number of range bins within the ROI that exceed the adaptive threshold are counted:

$$N_{active} = \sum (BF_{ROI} > T_{detect})$$

Where BF_{ROI} defines the signals for range bins within the ROI. A frame is then classified as active if:

$$N_{active} \geq N_{min}$$

where again, N_{min} defines the minimum number of required active bins to define a rat being present in the frame and was defined through testing and experimentation for the setup used in our project. For our project, we found that lower values of N_{min} accounting to approximately 5% of the total number of bins in the ROI proved most effective.

The next step of the algorithm was to apply the scoring system across a moving window of frames to reduce the number of false positives produced by noise spikes due to random noise. We made use of a deque to store a binary value of activity for each frame. That being, 1 for predicted rat presence and 0 otherwise for each radar frame. Then, the detection rate over this moving window is compted using:

$$R_{detect} = \frac{1}{N_{frames}} \sum_{i=1}^{N_{frames}} f_i$$

Where f_i is the binary classification for frame i. In our case, we took $N_{frames} = 10$ and found this to be a good balance between ignoring false positives without requiring motion for long durations from within the pipe. The classifier outputs a positive detection when

$$R_{detect} \geq \beta$$

where β defines the minimum active frames threshold which is again tuned to our setup. We found that β had to be tweaked for different materials and when using the radar in different environments i.e. different rooms. Overall, using this scoring system to aggregate a classification over time significantly improved the robustness of the classifier and ensured that detection of a rat had to persist over multiple frames before we output an alert to the user interface.

3.4 Visualisation and User Interface

The output from our software when in idle mode or when we do not detect an rodents can be seen in Figure 10. When a rat is detected, the user will observe something similar to 9. In both cases, the users have access to a live radar beamformed output with all the noise removal and filtering algorithms applied. Hence, we can see a clear depiction of the predicted rat location in the middle of the beamformed plot in 8. Furthermore, there is also a classification box, which outputs the result of the detected algorithm and displays "Jerry Detected!" in a red box, accompanied with the number of frames in which a rodent has been detected. When not detected or the amount of detection does not reach the threshold, a green box is displayed with corresponding confidence values.

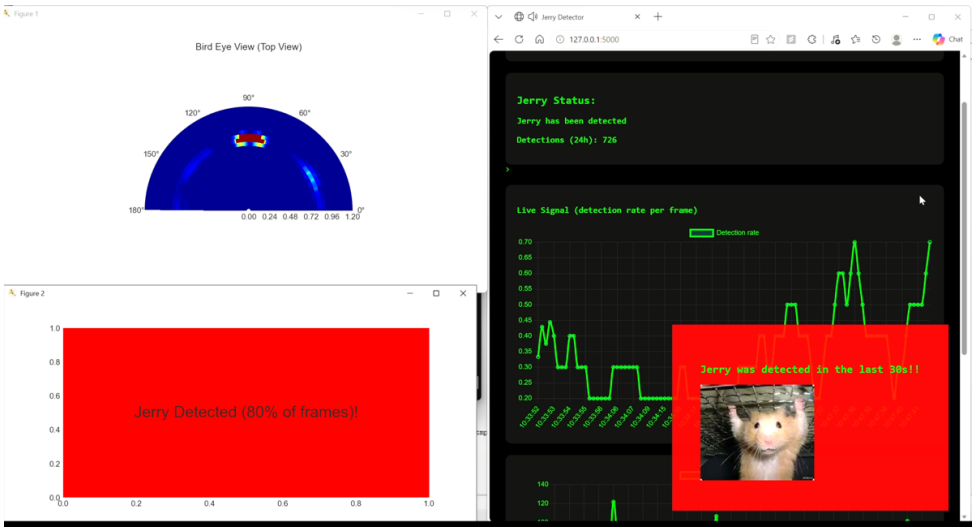
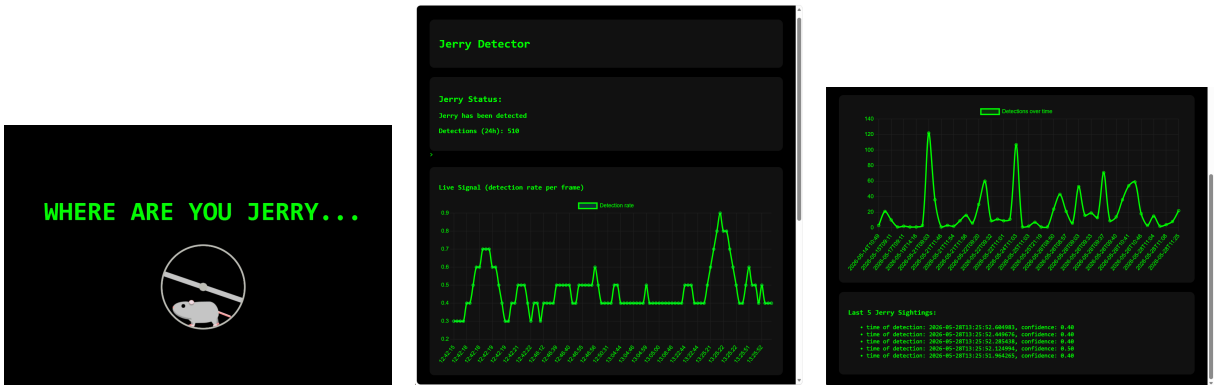


Figure 9: Example of the software output when a rat has been detected

As can also be seen, the users have access to a GUI window. This was designed with the idea that when multiple radars are distributed throughout a building, these site can act as a base station to track detections and monitor rat movement throughout the building. The GUI contains 5 main elements; A detection count to show the number of rodent detections in a 24 hour period; A plot of the streamed results from the classification algorithm showing when a rat was detected and for what percentage of the radar frames; A detection over time plot showing again live detection counts from the codebase; An alarm system which can be seen on the bottom right hand side of Figure 9 which indicated that the radar system has just detected a rat. Figure 8 also shows the GUI in more detail.



(a) Loading page of the GUI (b) Top of the main page (c) Top of the main page

Figure 8: Web page images highlighting the UI and plots showing live streamed data from the radar system

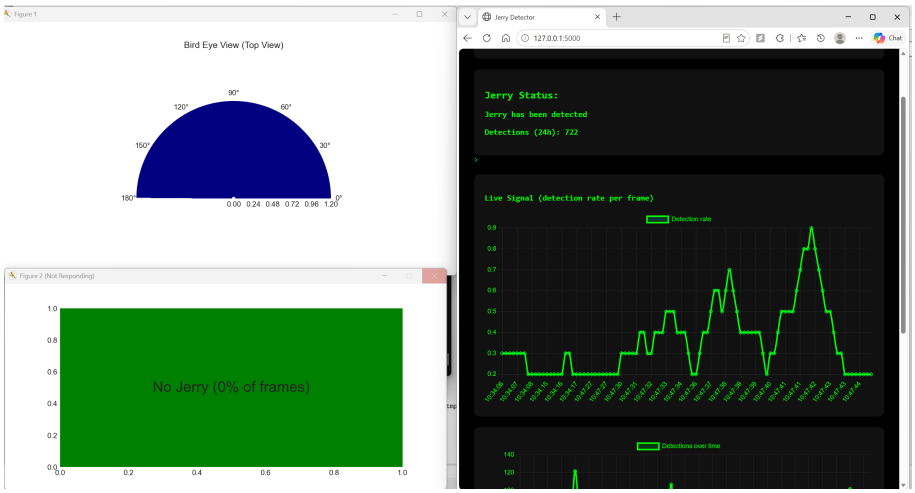


Figure 10: Example of the software output when no rat has been detected

4 Results

This project required rapid iteration and a "fail but fail quickly" ethos had to be applied. By the end of the course, we had achieved the deliverables we set out to achieve in the project plan and developed a system capable detecting a wind-up rat in a pipe through a simulated wall consisting of insulation foam and wood with consistency of correct detection in about 90% of tested setups. As can be seen in Figures 9 and 10, we have useful output and successful detection of a rat within a pipe network. We also developed dual radar compatibility with linux software which can be used by future groups who take the course and use linux machines. Aside from this, we also discovered some interesting results which are discussed below.

Also, as discussed in the project update, we encountered a strange phenomena when taking readings from the pipe when the dummy rodent was aligned with the radar. We note here that for the empty pipe, we struggle to see the wall behind the pipe. However, when implements are placed in the pipe, especially when such implement is aligned with the antenna array, we observe the wall behind strongly. We suggest this is due to a change in the propagation of the radar chirps when the pipe is filled versus when it is empty - there is possibly some destructive interference inside the empty pipe. This was unusual and appeared to not be so prominent when using a larger diameter pipe towards the end of the project.

One of the first area of testing was with the pipe away from the wall, versus the pipe fixed to the wall. In our testing, we found this did infact change the signal coming from the rat, but the wall was effected in a similar fashion in both cases. By that we mean both setups were differentiable and thus a rat was able to be classified in both cases.

The classification algorithm was unique to our project and was successful in classifying the rat moving through the pipe or shelf system. The pipes used where approximately 5mm thick and had a diameter of 10cm. The system also successfully detected the rat in cases where the pipe or shelf network was behind a wall. We tested 40mm thick insulation foam, 5mm thick wood and 5mm acrylic, as well as combinations of the three. We found that the classification algorithm, with tuning of thresholds to each setup (namely for setups involving wood, it required we drop our thresholds and sensitivity bounds as the material is less penetrable for the radar signals), again successfully classified the rat in all cases except those where acrylic was used.

We found this material prevented the radar signals from penetrating to detect the motion of the rat.

5 Issues

5.1 Linux-Setup

One of the primary issues appeared during the initial setup of the Linux architecture. This setup was based on the provided Mac-Linux tutorial, but it quickly became clear that the tutorial had not been fully validated for the Linux workflow. For instance, we were initially unable to flash the radars for Linux, even though this step was required before the Linux codebase could be used. All together, the first obstacle was not related to radar processing itself, but to making the basic acquisition infrastructure operational.

Reaching a working Linux setup required a considerable amount of debugging and reverse engineering. We had to identify which parts of the tutorial were applicable to Linux, which steps were specific to the Mac setup, and which assumptions were missing from the documentation. In parallel, we had to fix compatibility issues in the codebase and repeatedly test whether each modification brought the setup closer to a working state. This required understanding mechanisms at multiple levels, including the operating system, the acquisition hardware, the radar configuration, the DCA1000 streams, and finally the processing code. Since there was no validated reference setup to compare against, progress relied heavily on trial-and-error testing and inspection of errors at each stage.

Eventually, this process allowed us to reach a functional Linux setup: the radars could be flashed successfully, the codebase could be executed, and the acquisition pipeline could be used as a basis for further development. However, reaching this point required substantial effort before any meaningful dual-radar experimentation could begin. This made the setup phase one of the main unexpected difficulties of the project, as the provided resources served more as a starting point than as a directly usable or validated workflow.

5.2 Dual-Radar Setup

A subsequent difficulty appeared when we ported our Windows setup into the Linux setup. Our original pipeline had been built around assumptions specific to the Windows environment and to a single-radar workflow, which made it difficult to adapt directly. Several elements, such as configuration files, acquisition scripts, file paths, dependencies, data loading and recording, and stream management, had to be reconsidered for Linux.

Although the dual-radar setup eventually reached a functional state, the current processing pipeline still needs to be refined to produce more meaningful and reliable results.

During the live demo, we observed inconsistencies between the frames produced by radar 1 and radar 2, both before fusion in the individual polar maps and after fusion in the Cartesian map. We later found out the cause of this to be related to a bug caused by different configuration files used to start the radars and to process streaming. In other words, the radars were started using one configuration file, while processing relied on another. Resolving this issue resulted in the setup to produce more coherent results.

The feedback generated by our current setup is affected by an unresolved issue when a person is standing or moving in front of the radars. For instance, as shown in Figure 11 , while a person's movement produces a corresponding motion on the fused Cartesian map, this motion

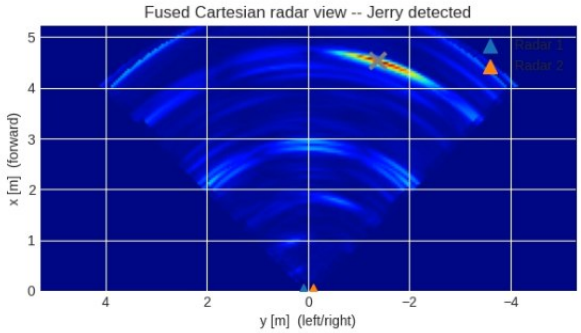


Figure 11: Detection associated with moving person appears as that of a farther "ghost" reflector.

is not perceived to originate from the person themselves but instead appears as an unknown object located further behind them. This behaviour had already been observed in previous experiments, but its underlying cause remains unexplained.

5.3 Noise Removal/Processing

DBSCAN clustering was also tested but in the end was not adopted due to its sensitivity to density variations in sparse radar point clouds, which can lead to excessive point rejection and potential loss of entire target structures under certain parameter settings. These findings were reflected in [8] who also found issues with point cloud removal rejecting the legs and heads of varying dog breeds. We investigated both GTrack and Kalman filters to see if they could help us locate the mouse better, but likely because of how small are target was, neither proved to be helpful with this setup.

Cosine ramps were also experimented with, but we found that infact a hamming window proved to be more effective for preprocessing in our case. Both techniques apply similar methodologies, so it is possible in circumstances with slightly noisier conditions that a cosine ramp would be more applicable.

5.4 Machine Learning Classification

Overall, using a machine learning model to detect rats did show promise but in our experimentation, generalised unreliably in new setups and thus we were not confident in using the models developed in our final system. We believe this is largely due to limited and insufficiently diverse input data. As discussed below, several methods were explored.

The simplest model we tried proved to be the most successful. We used logistic regression trained on features extracted from beamformed outputs and when trained and then evaluated on data we collected it achieved an 87% classifiation accuracy. While this indicates that there are learnable traits within the data we collected, these results didn't transfer when using the model for classification in real environments.

We also evaluated very simple Neural Networks and Convolutional Neural Networks on similarly gathered data and even with data augmentation to 5x our dataset size, we faced issues with overfitting and in the case of the CNN, the model learning to simply always output a classification of 1, as this was better than it guessing based on the parameters it was trained to learn from.

We finally tried to perform image classification based on beamform outputs, but the time taken to manually label a dataset of even 200 images was vast and hence it was decided that following a different approach was the mnost effective method to achieve our project goals.

Thus, given the project's emphasis on reliable and realtime operation, we pivoted to the classification algorithm defined previously. While possibly not as sophisticated, it was extremely robust and provided consistent behaviour in different test setups and conditions and required no retraining to do so. This alowed us to progress the pipeline and ensure other parameters could be tuned to their final values

5.5 Future Work

There are some areas that a group doing a similar project could focus on to further improve. Firstly, more experimentation with a dual radar setup would be the most useful area of investigation to seek the benefits of a second device for localisation and tracking techniques that we deemed unsuitable with a single radar setup.

With respect to the pre-processing pipeline, it could be beneficial to look into automatic wall ranging in order to define the distance to wall without manual input, allowing easy transition to different setups.

Similarly, further attempts could be made to gather more data with the hopes to improve a ML model such that it could generalise well to new scenarios and it wouldn't overfit to noise.

The current doppler detection worked well, but is only able to track radial velocity. When a mouse is moving perpendicular to the radar antenna, it's distance from the radar is not changing much, so it doesn't always appear clearly. Using two radars and angular velocity detection would potentially allow for much stronger tracking capability.

6 Code and Resources

6.1 Reference Material

As outlined in the introduction, many resources were used in the initial phases of the project to plan how we would approach the task. We also made use of further material throughout the project to help us attempt different techniques and to asses their suitability. For example, we used information detailed in[8] to help us recognise the using DBScan was very unsuitable for our use case. Furthermore, the provided background lectures and tutorial resources given by the COM-304 teaching team assisted us in understanding how the radars worked and proved to be a starting point for our development. So too, the project repository for "Realtime Human Tracking" [9] carried out by a group last year was a useful point of reference for us, especially with respect to the linux implementation.

6.2 Single Radar Repository

Our codebase which was used during the project demo can be found in the following GitHub repository[10]. In this repository, there is a detailed readme which highlights the structure of our repository and the purpose of all included files. There are also branches used for testing various parts of the pipeline which are further explained in the repository ReadMe.

6.3 Dual Radar Repository

Similarly to the above section, the codebase for the Dual-Radar Setup which was used during the project demo can be found in the following GitHub repository[11]. The ReadMe contains detailed explanation about the project structure and usage.

7 Individual Contributions

In general, the work was evenly distributed between team members. Collaboration between the 3 members allowed for rapid testing and iteration of the project throughout it's development in a relatively small timeframe. Due to the nature of the project, much of the time was dedicated to experimentation and this required the work and input of all members.

7.1 Contributions of Jan van der Kuyl

Jan was involved in the development of the windows pipeline and spent most of the time on the project working on this repository. This involved the initial porting across of tutorial code, refactoring this codebase to a minimum version suitable for our needs and forming the initial pipeline structure. His work also included testing various processing methods such as DBScan, GTrack and also different pipe configurations and setups to ensure the project moved in the right direction.

Jan also focussed on development and experimentation with respect to the classification algorithm. As highlighted previously in the progress report, large efforts were taken to develop a classifier using ML techniques. This involved lots of experimentation with different models and data gathering. However, after this proved ineffective, the group chose to pivot to a noise detection and recognition algorithm and Jan took the responsibility of designing and implementing this piece of the pipeline.

One of Jan's responsibilities was with respect to construction of the test stand. Previously, all members of the team had bought individual small components during active testing but Jan took on the responsibility of designing the test setup, sourcing and purchasing all the materials and organising their transportation back to campus. He also constructed the test stand with equipment, guidance and financial support from the SENS lab at EPFL [12].

The development of the GUI, while of low priority, was also taken on by Jan and the maintenance, cleanup and READMEs of the single radar repository were managed by him. He also took responsibility for organisation of the reports and completed final spell checking and reformatting of said reports for submission.

7.2 Contributions of Kasper Kelly

Kasper contributed with all tasks related to motion detection. He had to research potential methods, understand how the doppler effect could be used, and then how to implement it both in code and in testing. He built on this by improving on and adding various filtering methods, and adjusting lua configuration values and radar placement for the best results. This also included the addition of range gating, coupled with newer radar and pipe placement. Kasper also tested other processing methods that weren't successful for our scenario, like the use of GTrack and Kalman filters. In addition to just detection, he also helped design the testing

methods we used, including the use of Remi, a windup toy instead of pulling a plushy with a string.

7.3 Contributions of Kerim Tuncelli

Kerim contributed to the development and adaptation of the radar acquisition and processing pipeline. In the early stages of the project, he tuned the radar configurations to fit the initial project's needs and worked on refactoring parts of the codebase from the provided Hands-on Tutorial so that they could be used within the project's Windows setup. This involved adapting tutorial code that was originally designed for predefined tasks and restructuring it to fit the needs of their own experiments. This contributed to the first working version of the pipeline used to process radar data and visualize the results. He also enabled data collection for Task 3 and added support for experiment recording, which allowed them to save and analyse acquired radar data beyond live visualization.

After working on the Windows-based workflow, Kerim proceeded to set up the Linux-based acquisition and processing environment as the basis for the later dual-radar experiments. This required resolving issues related to the radar flashing and acquisition process, revising the provided Linux codebase to be functional, and building a framework to coordinate dependencies across the configuration files, the radar startup and streaming scripts, and the processing pipeline. Finally, he started porting the code of the final Windows pipeline onto the Linux setup.

The README of the Linux Dual Radar repository was managed by him.

References

[1] K. A. Byers, M. J. Lee, D. M. Patrick, and C. G. Himsworth, "Rats about town: A systematic review of rat movement in urban ecosystems," *Frontiers in Ecology and Evolution*, vol. 7, 2019.

[2] J. Hopkins, G. M. Santos-Elizondo, and F. Villablanca, "Detecting and monitoring rodents using camera traps and machine learning versus live trapping for occupancy modeling," *Frontiers in Ecology and Evolution*, vol. 12, 2024.

[3] T. Gabloffsky, K. Schuster, A. Zimmermann, A. Staffeld, A. Hawlitschka, R. Salomon, and L. Frintrop, "Establishment of an infrared-camera-based home-cage tracking system goblotrop," *eNeuro*, vol. 12, 2025.

[4] S.-C. Lai, S.-T. Wang, K.-L. Liu, and C.-Y. Wu, "A remote monitoring system for rodent infestation based on lorawan," *Sensors*, vol. 23, no. 9, 2023.

[5] T. Polonelli, M. Glahn, S. Kron, S. Selbert, M. Garzola, and M. Magno, "A non-invasive path to animal welfare: Contactless vital signs and activity monitoring of in-vivo rodents using a mm-wave fmcw radar," *arXiv*, 2025.

[6] L. Anishchenko, G. Gennarelli, A. Tataraidze, E. Gaysina, F. Soldovieri, and S. Ivashov, "Evaluation of rodents' respiratory activity using a bioradar," *IET Radar, Sonar & Navigation*, vol. 9, 2015.

[7] K. A. Clarke and A. J. Parker, "A quantitative study of normal locomotion in the rat," *Physiology Behavior*, vol. 38, no. 3, pp. 345–351, 1986.

[8] "Dbscan based clustering issues in point cloud denoising and structure preservation," 2024. Accessed: 2026-05-28.

[9] A. et al, "Real-time human tracking with mmwave radar." https://github.com/COM-304-Group-2/COM-304-Radars, 2026.

[10] "Com-304-rat-detection." https://github.com/Janvdk5/COM-304-Rat-Detection, 2026. GitHub repository, accessed 2026-05-29.

[11] "Com-304-rat-detection_linux." https://github.com/KerimTun/COM-304_Rat-Detection_LINUX, 2026. GitHub repository, accessed 2026-05-29.

[12] Laboratory of Sensing and Networking Systems, EPFL. https://sens.epfl.ch/, 2026.